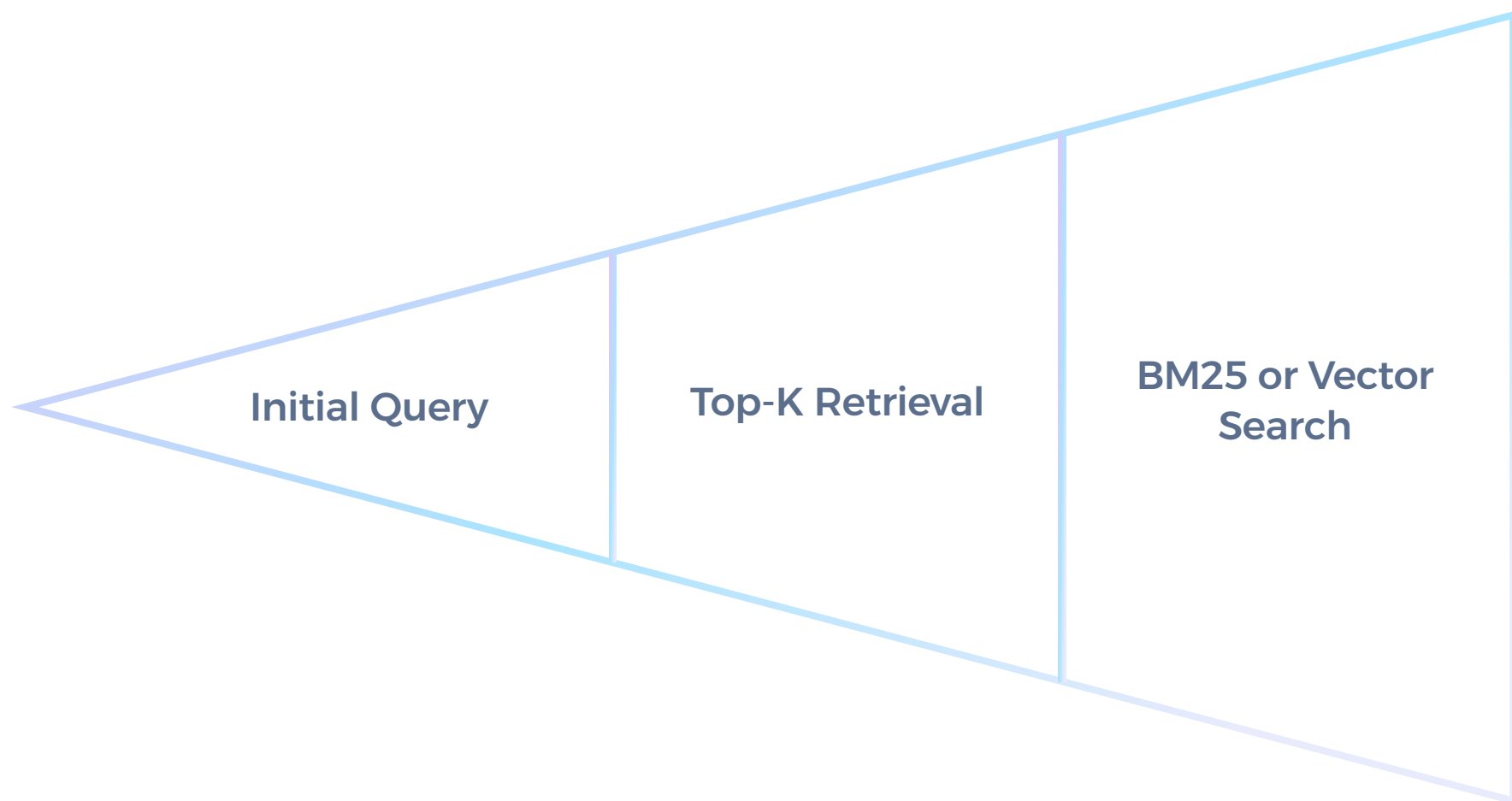


🔍 ADVANCED AI RETRIEVAL

CrossEncoder Reranking

A deep dive into the **Hybrid AI Search + Reranking** pipeline — combining Vector Search, BM25, and CrossEncoder AI to deliver smarter, more relevant search results.



This file belongs to the **Advanced AI Retrieval + Reranking Layer** — the intelligence engine behind high-quality document retrieval.

What This File Does

This is a **Hybrid AI Search + Reranking** file. It combines initial fast retrieval with deep AI-powered reranking to ensure the most relevant documents always surface at the top.



Vector Search

Semantic similarity search using embedding vectors for conceptual matching



BM25 Search

Keyword-based retrieval with field weighting for precise term matching



CrossEncoder Reranking

AI model that deeply compares query + document pairs to predict true relevance



Smarter Results

Final reordered output with rank change tracking and improvement metrics

High-Level Flow

The complete search pipeline flows through two distinct stages – fast retrieval followed by intelligent reranking.

01

User Enters Query

A natural language query is submitted via command line or API call

03

Retrieve Top-K Candidates

Collect the top candidate documents based on initial similarity scores

05

Deep Relevance Scoring

CrossEncoder compares each query-document pair and assigns new rerank scores

02

BM25 or Vector Search

Perform initial retrieval using keyword-based BM25 or semantic vector search

04

Send to CrossEncoder

Query + all retrieved documents are sent to the CrossEncoder AI model

06

Reorder & Return

Documents are sorted by rerank score and the best results are returned

Imports & Configuration

Import Section

```
import { MongoClient } from 'mongodb';  
import dotenv from 'dotenv';  
import axios from 'axios';
```

- **mongodb** → MongoDB Atlas connection
- **dotenv** → Load .env variables
- **axios** → API calls to AI services

API Configuration Constants

```
const TESTLEAF_API_BASE = ...  
const USER_EMAIL = ...  
const AUTH_TOKEN = ...  
const HF_API_KEY = ...
```

- **TESTLEAF_API_BASE** → Base URL for embedding API
- **USER_EMAIL / AUTH_TOKEN** → Authentication credentials
- **HF_API_KEY** → Hugging Face API key for CrossEncoder

 Environment variables are loaded via `dotenv.config()` from a `.env` file at runtime.

What Is CrossEncoder?

CrossEncoder is an **AI reranking model** that goes far beyond simple similarity scores – it reads the actual query and document together to predict true relevance.

Vector Search Approach

Compares Vectors

Converts query and document into embedding vectors, then measures cosine similarity between them

Fast but Approximate

Efficient at scale but may miss nuanced semantic relationships between query and content

CrossEncoder Approach

Reads Actual Pairs

Processes the full query + document text together as a single input to the transformer model

Slow but Accurate

Predicts relevance with much higher precision – ideal for reranking a small candidate set

Real-Time Example: Query: *"merge UHID issue"* – CrossEncoder understands that *"UHID merge validation"* is the most relevant document, even if vector similarity alone would rank it lower.

Core Functions Explained

1

generateQueryEmbedding()

Sends the user query to the Testleaf AI API via `axios.post()` and returns the embedding vector, token usage, and cost for monitoring.

2

vectorSearch()

Performs semantic vector search using MongoDB Atlas. Uses `numCandidates: Math.max(limit * 2, 100)` to control how many nearby vectors are checked. Supports optional dynamic filters and returns a `vectorSearchScore`.

3

bm25Search()

Performs keyword-based BM25 search with configurable field weights. Example: **title match** is weighted higher than **steps match** to prioritize the most relevant fields.

4

rerankWithCrossEncoder()

The **most important function**. Validates the Hugging Face API key, prepares document text, sends query + documents to the CrossEncoder model, and returns new relevance scores for intelligent reordering.

How CrossEncoder Works Internally

The CrossEncoder API call uses the Hugging Face model with two key inputs that define how relevance is computed.

API Input Structure

source_sentence: "merge UHID"

sentences: [

"verify patient UHID...",
"update patient record...",
"merge validation steps..."
]

source_sentence = original user query

sentences = all retrieved documents

What CrossEncoder Predicts

→ Reads Full Context

Processes query and each document together as a combined input – not separately

→ Assigns Relevance Score

Outputs a float score per document indicating how relevant it is to the query

→ Enables Reordering

Documents are sorted by new scores – truly relevant results rise to the top

✔ CrossEncoder compares: Query: "merge UHID" + Document: "verify patient UHID merge validation" → Predicts: **high relevance** ✓

Output, Comparison & Scoring

displayResults()

Prints search results in a structured format showing:

- **Title** of the matched document
- **Score** — original and rerank score
- **Module** — document category
- **Rank Changes** — how position shifted
- **Description** — document summary

displayComparison()

Shows a **Before vs After** reranking comparison so you can visually verify how much the AI improved result ordering.

Combined Score Object

```
return {  
  originalScore,  
  rerankScore,  
  rankChange,  
  improvementPct  
}
```

Timing Breakdown

Search Time

Time taken for BM25 or
Vector retrieval phase

Reranking Time

Time taken for
CrossEncoder scoring
phase

Total Time

End-to-end pipeline execution time

Running the File & Main Function

The file is executable as a Node.js script via the shebang line `#!/usr/bin/env node` and accepts command-line arguments to control the full search pipeline.

Command Line Example: `node rerank.js "merge UHID" vector 50 10`
Query = **merge UHID** | Search Type = **vector** | Retrieve top **50** | Return top **10**

1

Step 1

Initial Retrieval – fetch top candidate documents using BM25 or Vector Search

2

Step 2

CrossEncoder Reranking – generate improved relevance scores for all candidates

3

Step 3

Final Sort – reorder by rerank score and return the best results to the user

Cost Monitoring

- Embedding cost per query displayed
- Token usage tracked and reported

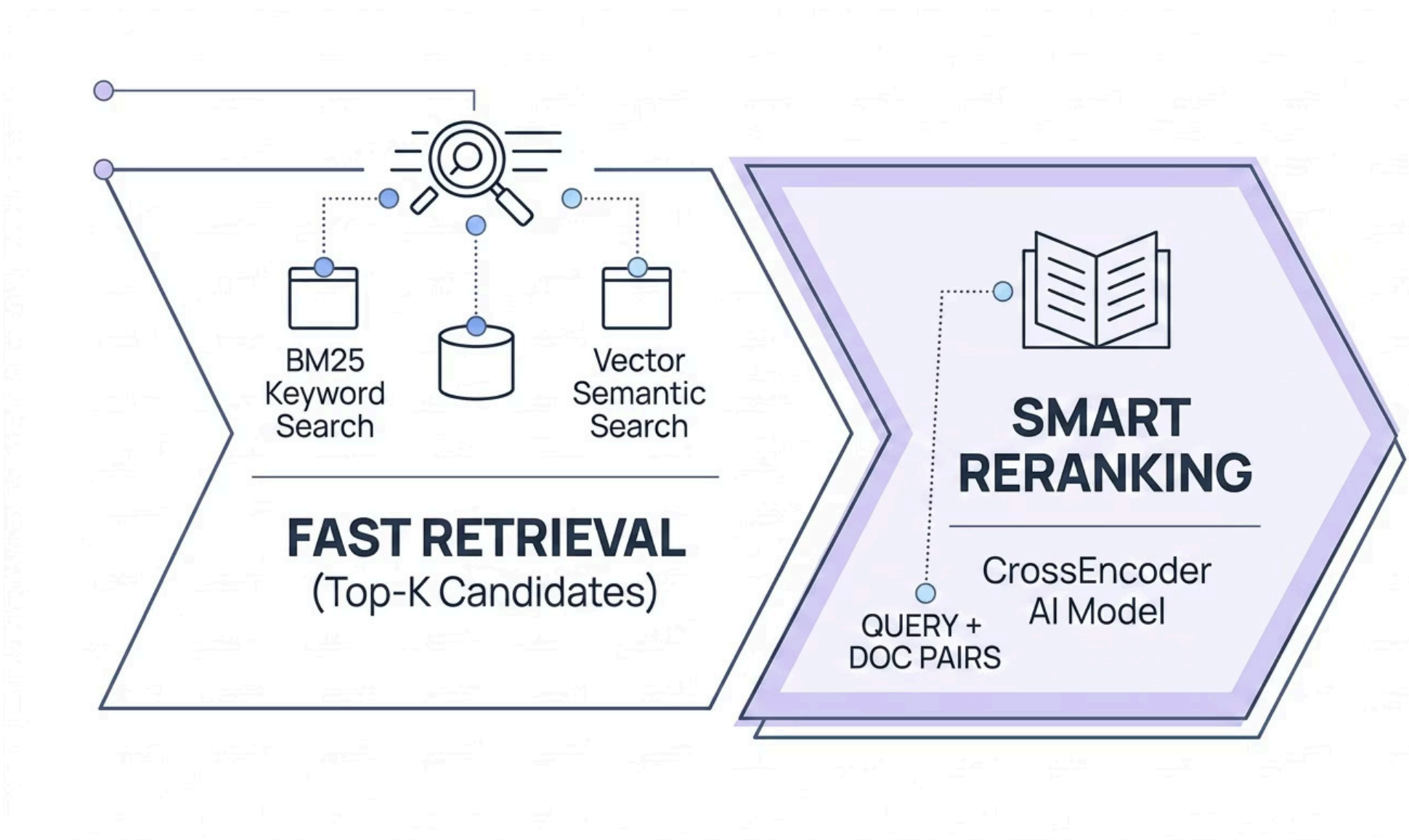
finally Block

```
finally {  
  await client.close();  
}
```

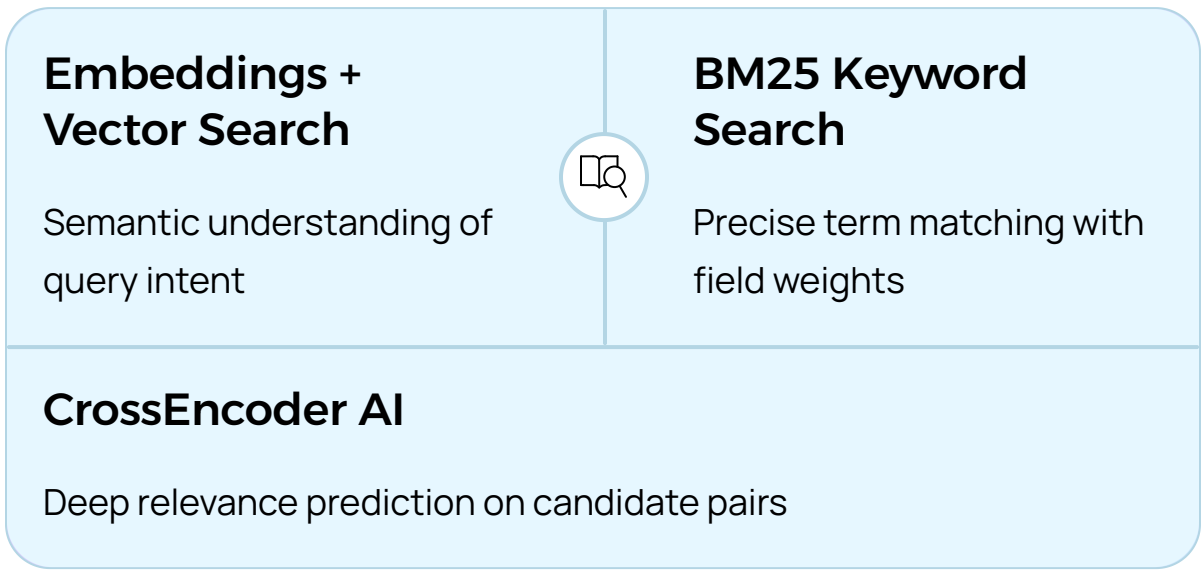
Always closes the MongoDB connection – even if an error occurs during execution.

Two-Stage Architecture & Why It Matters

This file implements a **Two-Stage Retrieval Architecture** — the gold standard for production AI search systems.



What This File Combines



What Happens Without This File

- Reranking becomes **unavailable**
- Search quality **decreases**
- Less relevant results appear at top
- Advanced AI retrieval pipeline **breaks**

⚠ This file is a **critical component** of the retrieval pipeline. Removing it degrades the entire search experience.